

山东大学

计算机科学与技术学院

2024－2025 学年第一学期

大作业实验报告

课程名称：　认知科学与类脑计算

实验名称：　基于 GPT-SoVITS 的语音生成

专　业　班　级　2022 级人工智能班

学　生　学　号　202200130077

学　生　姓　名　薛润泽

实　验　时　间　2024 年 11 月 23 日

一、实验目的

本实验旨在通过基于 GPT-SoVITS 项目的接口适配工作，深入了解认知科学与类脑计算在实际应用中的体现，特别是语音生成和转换技术。实验目标包括掌握 GPT-SoVITS 的工作原理，进行源码的拆分以及代码重写，接口适配（命令行可运行），图形化适配，并解决在适配过程中遇到的技术问题，从而加深对神经网络训练和接口开发的理解。

二、实验原理

GPT-SoVITS 是一种结合了 GPT 结构和 SoVITS（基于变分自编码器 VAE 和生成对抗网络 GAN 的歌唱声音转换技术）的语音生成系统。其原理主要包括：

1、使用低音质参考音频和对应的文本，通过 ASR（自动语音识别）技术转换为 token。

2、利用类 GPT 结构，根据目标文本和参考 token 生成补全的、包含参考音色的 token。

3、通过 SoVITS 模型，将补全的 token 转换为高音质音频。

此外，GPT-SoVITS-WebUI 提供了便捷的 Web 用户界面，支持零样本文本到语音（TTS）、少样本 TTS、跨语言支持等功能，以及声音伴奏分离、自动训练集分割等集成工具。

但是，由于 GPT-SoVITS 项目集成度过高，且没有提供各个功能的接口，所以在使用该项目组成其他项目时十分困难。

三、实验内容

1、对 GPT-SoVITS 项目进行接口适配，将原项目的多个功能拆分，并编写接口，使其可以通过命令行运行。在本实验中，对项目的声音伴奏分离功能、音频训练集分割功能、GPT 和 SoVITS 模型的训练和推理功能、文本语音推理功能等进行了拆分与重写，并使用 Control.py 文件控制总体流程。

2、取消 Gradio 对源代码进行的 Web 图形化，重写被 Gradio 内含的函数，并解决原函数参数匹配问题。取消项目图形化，实现命令行运行。

3、解决源代码在 Pytorch 架构下更换接口时遇到的本机环境问题。

4、验证分割后 GPT-SoVITS 的各项功能，包括声音伴奏分离、自动训练集分割、中文自动语音识别(ASR)和文本标注等。

5、训练分割后的 GPT-SoVITS，并尝试进行 TTS 推理。

四、实验要求

1、熟练掌握 GPT-SoVITS 的基本原理和工作流程。

2、成功完成 GPT-SoVITS 项目的接口适配工作，并通过命令行运行。

3、解构源代码，重写源代码各功能模块，并确保各项功能正常运行。

五、实验源程序

1、rmvbgm.py：用于声音伴奏分离。

2、slice_tool.py：用于自动训练集分割。

3、abctake.py、SoVITS_train.py、GPT_train.py：用于 GPT 和 SoVITS 模型的训练和推理。

4、tts4.py：用于文本语音推理，即用于文本到语音的转换。

5、Control.py：整合了上述所有 py 文件，控制训练总过程。

源代码具体内容如下：

Project.py（重新组合各模块可视化）：

```
import subprocess
powerShellCommand =  'python GPT-SoVITS-beta0706/control.py'
result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False, text=True)
import os
dir = "D:/大学/大三上/认知科学与类脑识别/大作业/GPT-SoVITS-beta0706"
os.chdir(dir)
import tkinter as tk
from tkinter import messagebox
import subprocess

def train_solve():
    powerShellCommand =  'runtime/python.exe rmvbgm.py cuda False D:/大学/大三上/认知科学与类脑识别/大作业/GPT-SoVITS-beta0706/tools/uvr5/uvr5_weights/HP2_all_vocals "D:/大学/大三上/认知科学与类脑识别/大作业/Ori_Audio" "D:/大学/大三上/认知科学与类脑识别/大作业/Solve_data/D-Vocal" "D:/大学/大三上/认知科学与类脑识别/大作业/Solve_data/D-BGM" 10 "flac"'
    result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False, text=True)
    powerShellCommand =  'runtime/python.exe slice_tool.py "D:/大学/大三上/认知科学与类脑识别/大作业/Solve_data/D-Vocal/vocal_Audio.mp3.reformatted.wav_10.flac" D:/大学/大三上/认知科学与类脑识别/大作业/Solve_data/Slice_Audio -34 4000 300 10 500 0.9 0.25 4'
    result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False, text=True)
    powerShellCommand =  'runtime/python.exe asr_solve.py D:/大学/大三上/认知科学与类脑识别/大作业/Solve_data/Slice_Audio D:/大学/大三上/认知科学与类脑识别/大作业/Solve_data/ASR_Audio "达摩 ASR (中文)" large zh'
    result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False, text=True)
    powerShellCommand =  'runtime/python.exe abctake.py --input_text D:/大学/大三上/认知科学与类脑识别/大作业/Solve_data/ASR_Audio/Slice_Audio.list --input_wav_dir D:/大学/大三上/认知科学与类脑
```

```
识别/大作业/Solve_data/Slice_Audio --exp_name Show_Project --gpu_numbers1a 0 --gpu_numbers1Ba 0
--gpu_numbers1c 0 --bert_pretrained_dir
GPT_SoVITS/pretrained_models/chinese-roberta-wwm-ext-large --ssl_pretrained_dir
GPT_SoVITS/pretrained_models/chinese-hubert-base --pretrained_s2g_path
GPT_SoVITS/pretrained_models/s1bert25hz-2kh-longer-epoch=68e-step=50232.ckpt --is_half'
    result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False,
text=True)
    powerShellCommand =  'runtime/python.exe SoVITS_train.py --batch_size 2 --total_epoch 8
--exp_name "Show_Project" --text_low_lr_rate 0.4 --if_save_latest 1 --if_save_every_weights 1
--save_every_epoch 4 --gpu_numbers "0" --pretrained_s2G
"GPT_SoVITS/pretrained_models/s2G488k.pth" --pretrained_s2D
"GPT_SoVITS/pretrained_models/s2D488k.pth"'
    result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False,
text=True)
    powerShellCommand =  'runtime/python.exe GPT_train.py --exp_name "Show_Project"
--pretrained_s1 "GPT_SoVITS/pretrained_models/s1bert25hz-2kh-longer-epoch=68e-step=50232.ckpt"
--gpu_numbers 0,1 --batch_size 4 --total_epoch 15'
    result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False,
text=True)
    print("All_Train_Over")

def tts_solve():
    input_text = text_entry.get()
    comd1 = 'D:/大学/大三上/认知科学与类脑识别/大作业/GPT-SoVITS-beta0706/runtime/python.exe
tts4.py  --ref_wav_path "D:/大学/大三上/认知科学与类脑识别/大作业
/Solve_data/D-Vocal/vocal_Audio.mp3.reformatted.wav_10.flac" --gpt_path "D:/大学/大三上/认知科学
与类脑识别/大作业/GPT-SoVITS-beta0706/GPT_weights/Show_Project-e15.ckpt" --sovits_path "D:/大学/
大三上/认知科学与类脑识别/大作业/GPT-SoVITS-beta0706/SoVITS_weights/Show_Project_e8_s368.pth"
--text '
    comd2 =  ' --text_language zh --prompt_language zh'
    # powerShellCommand =  'D:/大学/大三上/认知科学与类脑识别/大作业
/GPT-SoVITS-beta0706/runtime/python.exe tts4.py  --ref_wav_path "D:/大学/大三上/认知科学与类脑识
别/大作业/Solve_data/D-Vocal/vocal_Audio.mp3.reformatted.wav_10.flac" --gpt_path "D:/大学/大三上
/认知科学与类脑识别/大作业/GPT-SoVITS-beta0706/GPT_weights/Show_Project-e15.ckpt" --sovits_path
"D:/大学/大三上/认知科学与类脑识别/大作业
/GPT-SoVITS-beta0706/SoVITS_weights/Show_Project_e8_s368.pth" --text "哈啰哈啰笑东西"
--text_language zh --prompt_language zh'
    powerShellCommand = comd1 +'"'+ input_text +'"'+ comd2
    print(powerShellCommand)
    result = subprocess.run(["powershell", "-Command", powerShellCommand], capture_output=False,
text=True)

# 创建主窗口
root = tk.Tk()
```

```
root.title("GPT_SoVITS Improved Project by 薛润泽")

# 创建并放置按钮
process_button = tk.Button(root, text="训练模型", command = train_solve)
process_button.pack(pady=5)

# 创建并放置标签
label = tk.Label(root, text="生成语音文本")
label.pack(pady=5)

# 创建并放置文本框
text_entry = tk.Entry(root, width=40)
text_entry.pack(pady=5)

process_button = tk.Button(root, text="生成语音", command = tts_solve)
process_button.pack(pady=5)
# 运行主循环
root.mainloop()
```

rmvbgm.py:

```
import os
import traceback
import logging
import librosa
import ffmpeg
import soundfile as sf
import torch
import sys
from mdxnet import MDXNetDereverb
from vr import AudioPre, AudioPreDeEcho
#模型路径在 D:\大学\大三上\人工智能应用实践
\Project\Audio_trans\GPT-SoVITS-beta\GPT-SoVITS-beta0706\tools\uvr5\uvr5_weights
#待处理音频是 input，可以输入文件夹
#数字是人声提取的激进程度参数
#"flac"代表输出格式
#runtime/python.exe rmvbgm.py cpu False model_name "C:\path\to\input" "C:\path\to\output_vocal"
"C:\path\to\output_ins" 10 "flac"
#runtime/python.exe rmvbgm.py cuda False D:\大学\大三上\人工智能应用实践
\Project\Audio_trans\GPT-SoVITS-beta\GPT-SoVITS-beta0706\tools\uvr5\uvr5_weights\HP2_all_vocal
s "D:\大学\大三上\人工智能应用实践\Project\Audio_trans\GPT-SoVITS-beta\SHENLIvoice" "C:\Users\薛润
泽\Desktop\outputtest" "C:\Users\薛润泽\Desktop\outputtest" 10 "flac"

# 设置日志
```

```
logger = logging.getLogger(__name__)

# 权重路径
weight_uvr5_root = "tools/uvr5/uvr5_weights"
uvr5_names = []
for name in os.listdir(weight_uvr5_root):
    if name.endswith(".pth") or "onnx" in name:
        uvr5_names.append(name.replace(".pth", ""))

# 读取命令行参数
if len(sys.argv) < 7:
    print("用法: python script.py <device> <is_half> <model_name> <input_dir> <output_vocal_dir> <output_ins_dir> [<agg>] [<format>]")
    sys.exit(1)

device = sys.argv[1]
is_half = sys.argv[2]
model_name = sys.argv[3]
input_dir = sys.argv[4]
output_vocal_dir = sys.argv[5]
output_ins_dir = sys.argv[6]
agg = int(sys.argv[7]) if len(sys.argv) > 7 else 10  # 默认值为10
format0 = sys.argv[8] if len(sys.argv) > 8 else "flac"  # 默认格式为flac

def uvr(model_name, inp_root, save_root_vocal, save_root_ins, agg, format0):
    infos = []
    try:
        inp_root = inp_root.strip()
        save_root_vocal = save_root_vocal.strip()
        save_root_ins = save_root_ins.strip()

        if model_name == "onnx_dereverb_By_FoxJoy":
            pre_fun = MDXNetDereverb(15)
        else:
            func = AudioPre if "DeEcho" not in model_name else AudioPreDeEcho
            pre_fun = func(
                agg=int(agg),
                model_path=os.path.join(weight_uvr5_root, model_name + ".pth"),
                device=device,
                is_half=is_half,
            )

        paths = [os.path.join(inp_root, name) for name in os.listdir(inp_root)] if inp_root else []
```

```
        for path in paths:
            inp_path = os.path.join(inp_root, path)
            if not os.path.isfile(inp_path):
                continue
            need_reformat = 1
            done = 0
            try:
                info = ffmpeg.probe(inp_path)
                if ("channels" in info["streams"][0] and info["streams"][0]["channels"] == 2 and
info["streams"][0]["sample_rate"] == "44100"):
                    need_reformat = 0
                    pre_fun._path_audio_(inp_path, save_root_ins, save_root_vocal, format0)
                    done = 1
            except Exception as e:
                need_reformat = 1
                traceback.print_exc()

            if need_reformat == 1:
                tmp_path = os.path.join(os.environ["TEMP"],
f"{os.path.basename(inp_path)}.reformatted.wav")
                os.system(f"ffmpeg -i {inp_path} -vn -acodec pcm_s16le -ac 2 -ar 44100 {tmp_path}
-y")
                inp_path = tmp_path

            try:
                if done == 0:
                    pre_fun._path_audio_(inp_path, save_root_ins, save_root_vocal, format0)
                infos.append(f"{os.path.basename(inp_path)} -> Success")
            except Exception as e:
                infos.append(f"{os.path.basename(inp_path)} -> {traceback.format_exc()}")

    except Exception as e:
        infos.append(traceback.format_exc())

    finally:
        try:
            if model_name == "onnx_dereverb_By_FoxJoy":
                del pre_fun.pred.model
                del pre_fun.pred.model_
            else:
                del pre_fun.model
                del pre_fun
        except Exception as e:
```

```
            traceback.print_exc()

        print("clean_empty_cache")
        if torch.cuda.is_available():
            torch.cuda.empty_cache()

    return "\n".join(infos)

# 执行处理并输出结果
result = uvr(model_name, input_dir, output_vocal_dir, output_ins_dir, agg, format0)
print(result)
```

slice_tool.py（用于训练集长音频分割）：

```
import os
import sys
import numpy as np
from scipy.io import wavfile
from subprocess import Popen
from tools.my_utils import load_audio  ,clean_path
from slicer2 import Slicer

# 全局变量
ps_slice = []
python_exec = sys.executable  # 获取当前 Python 解释器的路径

def slice(inp, opt_root, threshold, min_length, min_interval, hop_size, max_sil_kept, _max, alpha,
i_part, all_part):
    os.makedirs(opt_root, exist_ok=True)

    if os.path.isfile(inp):
        input_files = [inp]
    elif os.path.isdir(inp):
        input_files = [os.path.join(inp, name) for name in sorted(os.listdir(inp))]
    else:
        return "输入路径存在但既不是文件也不是文件夹"

    slicer = Slicer(
        sr=32000,  # 长音频采样率
        threshold=int(threshold),  # 音量小于这个值视作静音的备选切割点
        min_length=int(min_length),  # 每段最小多长
        min_interval=int(min_interval),  # 最短切割间隔
```

```
        hop_size=int(hop_size),  # 音量曲线计算精度
        max_sil_kept=int(max_sil_kept),  # 切完后静音最多留多长
    )

    _max = float(_max)
    alpha = float(alpha)

    for inp_path in input_files[int(i_part)::int(all_part)]:
        try:
            name = os.path.basename(inp_path)
            audio = load_audio(inp_path, 32000)
            for chunk, start, end in slicer.slice(audio):  # start 和 end 是帧数
                tmp_max = np.abs(chunk).max()
                if tmp_max > 1:
                    chunk /= tmp_max
                chunk = (chunk / tmp_max * (_max * alpha)) + (1 - alpha) * chunk
                wavfile.write(
                    f"{opt_root}/{name}_{start:010d}_{end:010d}.wav",
                    32000,
                    (chunk * 32767).astype(np.int16),
                )
        except Exception as e:
            print(inp_path, "->fail->", e)

    return "执行完毕，请检查输出文件"

# ps_slice=[]
# def
open_slice(inp,opt_root,threshold,min_length,min_interval,hop_size,max_sil_kept,_max,alpha,n_p
arts):
#     global ps_slice
#     inp = my_utils.clean_path(inp)
#     opt_root = my_utils.clean_path(opt_root)
#     if(os.path.exists(inp)==False):
#         yield "输入路径不存在
",{"__type__":"update","visible":True},{"__type__":"update","visible":False}
#         return
#     if os.path.isfile(inp):n_parts=1
#     elif os.path.isdir(inp):pass
#     else:
#         yield "输入路径存在但既不是文件也不是文件夹
",{"__type__":"update","visible":True},{"__type__":"update","visible":False}
#         return
#     if (ps_slice == []):
```

```
#         for i_part in range(n_parts):
#             cmd = '"%s" tools/slice_audio.py "%s" "%s" %s %s %s %s %s %s %s %s %s''' %
(python_exec,inp, opt_root, threshold, min_length, min_interval, hop_size, max_sil_kept, _max,
alpha, i_part, n_parts)
#             print(cmd)
#             p = Popen(cmd, shell=True)
#             ps_slice.append(p)
#         yield "切割执行中", {"__type__": "update", "visible": False}, {"__type__": "update",
"visible": True}
#         for p in ps_slice:
#             p.wait()
#         ps_slice=[]
#         yield "切割结束
",{"__type__":"update","visible":True},{"__type__":"update","visible":False}
#     else:
#         yield "已有正在进行的切割任务，需先终止才能开启下一次任务", {"__type__": "update", "visible":
False}, {"__type__": "update", "visible": True}

def open_slice(inp, opt_root, threshold, min_length, min_interval, hop_size, max_sil_kept, _max,
alpha, n_parts):
    # print("INTTTTTTTTTTTTT")
    global ps_slice
    # inp = load_audio.clean_path(inp)
    # opt_root = load_audio.clean_path(opt_root)
    print('okkkkkkkk',inp)
    inp = clean_path(inp)
    print('IIIIIIINNNNNNNN',inp)
    opt_root = clean_path(opt_root)

    if not ps_slice:
        for i_part in range(int(n_parts)):
            # cmd = f'"{python_exec}" {os.path.abspath(__file__)} slice "{inp}" "{opt_root}"
{threshold} {min_length} {min_interval} {hop_size} {max_sil_kept} {_max} {alpha} {i_part}
{n_parts}'
            # cmd = f' "{inp}" "{opt_root}" {threshold} {min_length} {min_interval} {hop_size}
{max_sil_kept} {_max} {alpha} {i_part} {n_parts}  '
            cmd = '"%s" tools/slice_audio.py "%s" "%s" %s %s %s %s %s %s %s %s %s''' % (python_exec,inp,
opt_root, threshold, min_length, min_interval, hop_size, max_sil_kept, _max, alpha, i_part,
n_parts)
            # print(cmd)
            p = Popen(cmd, shell=True)
            ps_slice.append(p)

        for p in ps_slice:
```

```
            p.wait()

        ps_slice.clear()
        return "切割结束"
    else:
        return "已有正在进行的切割任务"

if __name__ == "__main__":
    # print('SSSSSsys', sys.argv[1:])
    result = open_slice(*sys.argv[1:])
    print(result)
```

abctake.py（预训练处理）：

```
import os
import argparse
import argparse
import traceback
from subprocess import Popen
from my_utils import clean_path  # 假设 my_utils 模块中已经定义了 clean_path 函数
from config import
python_exec,infer_device,is_half,exp_root,webui_port_main,webui_port_infer_tts,webui_port_uvr5,
webui_port_subfix,is_share

# 假设以下全局变量已经在其他地方定义
is_half = True  # 是否使用半精度浮点数
ps1abc=[]
def open1abc(inp_text, inp_wav_dir, exp_name, gpu_numbers1a, gpu_numbers1Ba, gpu_numbers1c,
bert_pretrained_dir, ssl_pretrained_dir, pretrained_s2G_path):
    global ps1abc  # 假设 ps1abc 是一个全局列表，用于存储进程对象
    inp_text = clean_path(inp_text)
    inp_wav_dir = clean_path(inp_wav_dir)
    if ps1abc == []:
        opt_dir = f"{exp_root}/{exp_name}"
        try:
            # ############################## 1a
            path_text = f"{opt_dir}/2-name2text.txt"
            if not os.path.exists(path_text) or (os.path.exists(path_text) and len(open(path_text,
"r", encoding="utf8").read().strip("\n").split("\n")) < 2):
                config = {
                    "inp_text": inp_text,
                    "inp_wav_dir": inp_wav_dir,
                    "exp_name": exp_name,
```

```
            "opt_dir": opt_dir,
            "bert_pretrained_dir": bert_pretrained_dir,
            "is_half": str(is_half)
        }
        gpu_names = gpu_numbers1a.split("-")
        all_parts = len(gpu_names)
        for i_part in range(all_parts):
            config.update({
                "i_part": str(i_part),
                "all_parts": str(all_parts),
                "_CUDA_VISIBLE_DEVICES": gpu_names[i_part],
            })
            os.environ.update(config)
            cmd = f'"{python_exec}" GPT_SoVITS/prepare_datasets/1-get-text.py'
            print(cmd)
            p = Popen(cmd, shell=True)
            ps1abc.append(p)
        yield "进度：1a-ing", {"__type__": "update", "visible": False}
        for p in ps1abc:
            p.wait()

        opt = []
        for i_part in range(all_parts):
            txt_path = f"{opt_dir}/2-name2text-{i_part}.txt"
            with open(txt_path, "r", encoding="utf8") as f:
                opt += f.read().strip("\n").split("\n")
            os.remove(txt_path)
        with open(path_text, "w", encoding="utf8") as f:
            f.write("\n".join(opt) + "\n")
        assert len("".join(opt)) > 0, "1Aa-文本获取进程失败"
        yield "进度：1a-done", {"__type__": "update", "visible": True}
ps1abc = []
# ############################## 1b
config = {
    "inp_text": inp_text,
    "inp_wav_dir": inp_wav_dir,
    "exp_name": exp_name,
    "opt_dir": opt_dir,
    "cnhubert_base_dir": ssl_pretrained_dir,
}
gpu_names = gpu_numbers1Ba.split("-")
all_parts = len(gpu_names)
for i_part in range(all_parts):
    config.update({
```

```
                    "i_part": str(i_part),
                    "all_parts": str(all_parts),
                    "_CUDA_VISIBLE_DEVICES": gpu_names[i_part],
                })
                os.environ.update(config)
                cmd = f'"{python_exec}" GPT_SoVITS/prepare_datasets/2-get-hubert-wav32k.py'
                print(cmd)
                p = Popen(cmd, shell=True)
                ps1abc.append(p)
            yield "进度：1a-done, 1b-ing",{"__type__": "update", "visible": True}
            for p in ps1abc:
                p.wait()
            yield "进度：1a1b-done", {"__type__": "update", "visible": True}
            ps1abc = []
            # ############################## 1c
            path_semantic = f"{opt_dir}/6-name2semantic.tsv"
            if not os.path.exists(path_semantic) or (os.path.exists(path_semantic) and
os.path.getsize(path_semantic) < 31):
                config = {
                    "inp_text": inp_text,
                    "exp_name": exp_name,
                    "opt_dir": opt_dir,
                    "pretrained_s2G": pretrained_s2G_path,
                    "s2config_path": "GPT_SoVITS/configs/s2.json",
                }
                gpu_names = gpu_numbers1c.split("-")
                all_parts = len(gpu_names)
                for i_part in range(all_parts):
                    config.update({
                        "i_part": str(i_part),
                        "all_parts": str(all_parts),
                        "_CUDA_VISIBLE_DEVICES": gpu_names[i_part],
                    })
                    os.environ.update(config)
                    cmd = f'"{python_exec}" GPT_SoVITS/prepare_datasets/3-get-semantic.py'
                    print(cmd)
                    p = Popen(cmd, shell=True)
                    ps1abc.append(p)
                yield "进度：1a1b-done, 1cing",  {"__type__": "update", "visible": True}
                for p in ps1abc:
                    p.wait()

                opt = ["item_name\tsemantic_audio"]
                for i_part in range(all_parts):
```

```
                    semantic_path = f"{opt_dir}/6-name2semantic-{i_part}.tsv"
                    with open(semantic_path, "r", encoding="utf8") as f:
                        opt += f.read().strip("\n").split("\n")
                    os.remove(semantic_path)
                with open(path_semantic, "w", encoding="utf8") as f:
                    f.write("\n".join(opt) + "\n")
                yield "进度：all-done",  {"__type__": "update", "visible": True}
            ps1abc = []
            yield "一键三连进程结束",{"__type__": "update", "visible": False}
        except Exception as e:
            traceback.print_exc()
            close1abc()  # 假设 close1abc 函数已经定义
            yield "一键三连中途报错", {"__type__": "update", "visible": False}
    else:
        yield "已有正在进行的一键三连任务，需先终止才能开启下一次任务",  {"__type__": "update",
"visible": True}

# 假设 close1abc 函数定义如下
def close1abc():
    global ps1abc
    if ps1abc != []:
        for p1abc in ps1abc:
            try:
                p1abc.terminate()  # 终止进程
            except:
                pass
        ps1abc = []
    return "已终止所有一键三连进程", {"__type__": "update", "visible": True}, {"__type__": "update",
"visible": False}

if __name__ == "__main__":
    # 创建解析器对象
    parser = argparse.ArgumentParser(description="一键三连工具：文本获取、SSL 特征提取、语义 token
提取")

    # 添加参数
    parser.add_argument("--input_text", type=str, required=True, help="输入文本文件的路径")
    parser.add_argument("--input_wav_dir", type=str, required=True, help="输入 WAV 音频文件的目录
")
    parser.add_argument("--exp_name", type=str, required=True, help="实验名称")
    parser.add_argument("--gpu_numbers1a", type=str, required=True, help="用于文本获取的 GPU 卡号，
以'-'分隔")
    parser.add_argument("--gpu_numbers1Ba", type=str, required=True, help="用于 SSL 特征提取的 GPU
卡号，以'-'分隔")
```

```
    parser.add_argument("--gpu_numbers1c", type=str, required=True, help="用于语义 token 提取的 GPU 卡号，以'-'分隔")
    parser.add_argument("--bert_pretrained_dir", type=str, required=True, help="预训练的 BERT 模型目录")
    parser.add_argument("--ssl_pretrained_dir", type=str, required=True, help="预训练的 SSL 模型目录")
    parser.add_argument("--pretrained_s2g_path", type=str, required=True, help="预训练的 S2G 模型路径")
    parser.add_argument("--exp_root", type=str, default=exp_root, help="实验根目录（可选）")
    parser.add_argument("--is_half", action='store_true', help="是否使用半精度浮点数（默认不使用）")

    # 解析参数
    args = parser.parse_args()

    # 设置参数值
    input_text_path = args.input_text
    input_wav_dir = args.input_wav_dir
    experiment_name = args.exp_name
    gpu_numbers1a = args.gpu_numbers1a
    gpu_numbers1Ba = args.gpu_numbers1Ba
    gpu_numbers1c = args.gpu_numbers1c
    bert_pretrained_directory = args.bert_pretrained_dir
    ssl_pretrained_directory = args.ssl_pretrained_dir
    pretrained_s2g_path = args.pretrained_s2g_path
    if args.exp_root:
        exp_root = args.exp_root
    is_half = args.is_half

    # 调用 open1abc 函数
    for message, updates in open1abc(input_text_path, input_wav_dir, experiment_name, gpu_numbers1a, gpu_numbers1Ba, gpu_numbers1c, bert_pretrained_directory, ssl_pretrained_directory, pretrained_s2g_path):
        # 在这里处理 open1abc 函数生成的消息和更新
        print(message)
```

SoVITS_train.py（SoVITS 模型训练）：

```
import os,sys
now_dir = os.getcwd()
sys.path.insert(0, now_dir)
import json
from subprocess import Popen
```

```
import argparse
from config import
python_exec,infer_device,is_half,exp_root,webui_port_main,webui_port_infer_tts,webui_port_uvr5,
webui_port_subfix,is_share

# 全局变量
SoVITS_weight_root = "SoVITS_weights"  # 请根据实际情况修改
tmp = os.path.join(now_dir, "TEMP")
# os.makedirs(tmp, exist_ok=True)

# 定义训练函数
p_train_SoVITS = None
def open1Ba(batch_size, total_epoch, exp_name, text_low_lr_rate,
            if_save_latest, if_save_every_weights, save_every_epoch,
            gpu_numbers1Ba, pretrained_s2G, pretrained_s2D):
    global p_train_SoVITS
    print("IN OPEN1BA")
    if p_train_SoVITS is None:
        with open("GPT_SoVITS/configs/s2.json") as f:
            data=f.read()
            data=json.loads(data)
            # data = json.load(f)

        s2_dir = f"{exp_root}/{exp_name}"
        os.makedirs(f"{s2_dir}/logs_s2", exist_ok=True)
        # 检查是否启用半精度
        if 'is_half' not in globals() or not is_half:
            data["train"]["fp16_run"] = False
            batch_size = max(1, batch_size // 2)

        # 配置设置
        data["train"]["batch_size"] = batch_size
        data["train"]["epochs"] = total_epoch
        data["train"]["text_low_lr_rate"] = text_low_lr_rate
        data["train"]["pretrained_s2G"] = pretrained_s2G
        data["train"]["pretrained_s2D"] = pretrained_s2D
        data["train"]["if_save_latest"] = if_save_latest
        data["train"]["if_save_every_weights"] = if_save_every_weights
        data["train"]["save_every_epoch"] = save_every_epoch
        data["train"]["gpu_numbers"] = gpu_numbers1Ba
        data["data"]["exp_dir"] = data["s2_ckpt_dir"] = s2_dir
        data["save_weight_dir"] = SoVITS_weight_root
        data["name"] = exp_name
```

```
        tmp_config_path="%s/tmp_s2.json"%tmp
        with open(tmp_config_path, "w") as f:
            f.write(json.dumps(data))
        cmd = '"%s" GPT_SoVITS/s2_train.py --config "%s"'%(python_exec,tmp_config_path)
        # cmd = f'"{python_exec}" GPT_SoVITS/s2_train.py --config "{tmp_config_path}"'

        with open(tmp_config_path,"w")as f:f.write(json.dumps(data))


        print(f"SoVITS 训练开始：{cmd}")
        p_train_SoVITS = Popen(cmd, shell=True)
        p_train_SoVITS.wait()
        p_train_SoVITS = None
        print("SoVITS 训练完成")
    else:
        print("已有正在进行的 SoVITS 训练任务，需先终止才能开启下一次任务")

# 主函数
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="SoVITS 训练参数")
    parser.add_argument("--batch_size", type=int, default=64, help="每批次的样本数")
    parser.add_argument("--total_epoch", type=int, default=100, help="总训练轮数")
    parser.add_argument("--exp_name", type=str, default="example_experiment", help="实验名称")
    parser.add_argument("--text_low_lr_rate", type=float, default=0.1, help="文本学习率")
    parser.add_argument("--if_save_latest", type=int, default=1, help="是否保存最新模型")
    parser.add_argument("--if_save_every_weights", type=int, default=1, help="是否每轮保存模型")
    parser.add_argument("--save_every_epoch", type=int, default=5, help="每隔多少轮保存一次模型")
    parser.add_argument("--gpu_numbers", type=str, default="0,1", help="可用 GPU 编号")
    parser.add_argument("--pretrained_s2G", type=str, default="path/to/pretrained/model_G.pth",
help="生成模型路径")
    parser.add_argument("--pretrained_s2D", type=str, default="path/to/pretrained/model_D.pth",
help="判别模型路径")

    args = parser.parse_args()
    # # 调用训练函数
    print("STARTTTTTTTTTTTTTTTT")
    open1Ba(args.batch_size, args.total_epoch, args.exp_name, args.text_low_lr_rate,
            args.if_save_latest, args.if_save_every_weights, args.save_every_epoch,
            args.gpu_numbers, args.pretrained_s2G, args.pretrained_s2D)
    print("OVEVRRRRRRRRRRRRRRR")
```

GPT_train.py（用于 GPT 模型训练）：

```
import os
```

```
import sys
import json
import yaml
from subprocess import Popen
import argparse
from config import python_exec, exp_root, is_half

# 全局变量
GPT_weight_root = "GPT_weights"  # 请根据实际情况修改
tmp = os.path.join(os.getcwd(), "TEMP")
os.makedirs(tmp, exist_ok=True)

# 定义 GPT 训练函数
p_train_GPT = None
def open1Bb(batch_size, total_epoch, exp_name, if_dpo, if_save_latest,
            if_save_every_weights, save_every_epoch, gpu_numbers, pretrained_s1):
    global p_train_GPT
    if p_train_GPT is None:
        with open("GPT_SoVITS/configs/s1longer.yaml") as f:
            data = yaml.safe_load(f)

        s1_dir = f"{exp_root}/{exp_name}"
        os.makedirs(f"{s1_dir}/logs_s1", exist_ok=True)

        # 检查是否启用半精度
        if not is_half:
            data["train"]["precision"] = "32"
            batch_size = max(1, batch_size // 2)

        # 配置设置
        data["train"]["batch_size"] = batch_size
        data["train"]["epochs"] = total_epoch
        data["pretrained_s1"] = pretrained_s1
        data["train"]["save_every_n_epoch"] = save_every_epoch
        data["train"]["if_save_every_weights"] = if_save_every_weights
        data["train"]["if_save_latest"] = if_save_latest
        data["train"]["if_dpo"] = if_dpo
        data["train"]["half_weights_save_dir"] = GPT_weight_root
        data["train"]["exp_name"] = exp_name
        data["train_semantic_path"] = f"{s1_dir}/6-name2semantic.tsv"
        data["train_phoneme_path"] = f"{s1_dir}/2-name2text.txt"
        data["output_dir"] = f"{s1_dir}/logs_s1"

        # 设置 GPU
```

```
        os.environ["_CUDA_VISIBLE_DEVICES"] = gpu_numbers
        os.environ["hz"] = "25hz"

        # 保存临时配置文件
        tmp_config_path = f"{tmp}/tmp_s1.yaml"
        with open(tmp_config_path, "w") as f:
            yaml.dump(data, f, default_flow_style=False)

        # 启动训练
        cmd = f'"{python_exec}" GPT_SoVITS/s1_train.py --config_file "{tmp_config_path}"'
        print(f"GPT 训练开始：{cmd}")
        p_train_GPT = Popen(cmd, shell=True)
        p_train_GPT.wait()
        p_train_GPT = None
        print("GPT 训练完成")
    else:
        print("已有正在进行的 GPT 训练任务，需先终止才能开启下一次任务")

# 主函数
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="GPT 训练参数")
    parser.add_argument("--batch_size", type=int, default=4, help="每批次的样本数")
    parser.add_argument("--total_epoch", type=int, default=15, help="总训练轮数")
    parser.add_argument("--exp_name", type=str, required=True, help="实验名称")
    parser.add_argument("--if_dpo", type=int, default=0, help="是否启用 DPO 训练")
    parser.add_argument("--if_save_latest", type=int, default=1, help="是否保存最新模型")
    parser.add_argument("--if_save_every_weights", type=int, default=1, help="是否每轮保存模型")
    parser.add_argument("--save_every_epoch", type=int, default=5, help="每隔多少轮保存一次模型")
    parser.add_argument("--gpu_numbers", type=str, default="0", help="可用 GPU 编号，用逗号分隔")
    parser.add_argument("--pretrained_s1", type=str, required=True, help="预训练 GPT 模型路径")

    args = parser.parse_args()

    # 调用 GPT 训练函数
    open1Bb(args.batch_size, args.total_epoch, args.exp_name, args.if_dpo,
            args.if_save_latest, args.if_save_every_weights, args.save_every_epoch,
            args.gpu_numbers, args.pretrained_s1)
```

tts4.py（模型训练完毕后用于语音合成推理）：

```
'''
按中英混合识别
按日英混合识别
多语种启动切分识别语种
```

```
全部按中文识别
全部按英文识别
全部按日文识别
'''
import os, re, logging
import LangSegment
logging.getLogger("markdown_it").setLevel(logging.ERROR)
logging.getLogger("urllib3").setLevel(logging.ERROR)
logging.getLogger("httpcore").setLevel(logging.ERROR)
logging.getLogger("httpx").setLevel(logging.ERROR)
logging.getLogger("asyncio").setLevel(logging.ERROR)
logging.getLogger("charset_normalizer").setLevel(logging.ERROR)
logging.getLogger("torchaudio._extension").setLevel(logging.ERROR)
import pdb
import torch
import argparse
import struct
def write_wav_file(filename, sampling_rate, data):
    """
    将音频数据写入 WAV 文件。

    参数:
    filename (str): 输出文件的路径和名称。
    sampling_rate (int): 音频的采样率。
    data (numpy.ndarray): 音频数据，应为整数类型（如 np.int16）。
    """
    with open(filename, 'wb') as f:
        # 写入 WAV 文件头（RIFF 块）
        f.write(b'RIFF')               # ChunkID
        f.write(struct.pack('<I', 36 + len(data) * 2))  # ChunkSize (36 for the rest of the header
+ data size)
        f.write(b'WAVE')              # Format
        # 写入 fmt 子块
        f.write(b'fmt ')              # Subchunk1ID
        f.write(struct.pack('<I', 16))        # Subchunk1Size
        f.write(struct.pack('<H', 1))         # AudioFormat (1 for PCM)
        f.write(struct.pack('<H', 1))         # NumChannels
        f.write(struct.pack('<I', sampling_rate))  # SampleRate
        f.write(struct.pack('<I', sampling_rate * 2))  # ByteRate (SampleRate * NumChannels *
BytesPerSample)
        f.write(struct.pack('<H', 2))         # BlockAlign (NumChannels * BytesPerSample)
        f.write(struct.pack('<H', 16))        # BitsPerSample
        # 写入 data 块头
        f.write(b'data')              # Subchunk2ID
```

```
        f.write(struct.pack('<I', len(data) * 2))  # Subchunk2Size
        # 写入音频数据
        f.write(data.tobytes())
parser = argparse.ArgumentParser(description='Text-to-Speech using GPT-SoVITS')
parser.add_argument('--ref_wav_path', type=str, help='Path to the reference audio file')
parser.add_argument('--prompt_language', type=str, required=False, help='Reference text language for the audio')
parser.add_argument('--prompt_text', type=str, required=False, help='Reference text for the audio')
parser.add_argument('--text', type=str, required=True, help='Text to synthesize')
# parser.add_argument('--text_language', type=str, choices=['zh', 'en', 'ja', 'all_zh', 'all_ja', 'auto'] ,help='Language of the input text')
parser.add_argument('--text_language', type=str ,help='Language of the input text')
parser.add_argument('--how_to_cut', type=str, choices=['none', 'sentences_4', 'chars_50', 'chinese_period', 'english_period', 'punctuation'], default='sentences_4', help='How to split the input text')
parser.add_argument('--top_k', type=int, default=20, help='Top-k sampling parameter')
parser.add_argument('--top_p', type=float, default=0.6, help='Top-p sampling parameter')
parser.add_argument('--temperature', type=float, default=0.6, help='Temperature parameter')
parser.add_argument('--ref_free', action='store_true', help='Enable reference-free mode')
parser.add_argument('--gpt_path', type=str,
default='GPT_SoVITS/pretrained_models/s1bert25hz-2kh-longer-epoch=68e-step=50232.ckpt',
help='Path to the GPT model weights')
parser.add_argument('--sovits_path', type=str,
default='GPT_SoVITS/pretrained_models/s2G488k.pth', help='Path to the SoVITS model weights')
parser.add_argument('--cnhubert_base_path', type=str,
default='GPT_SoVITS/pretrained_models/chinese-hubert-base', help='Path to the CNHubert model base')
parser.add_argument('--bert_path', type=str,
default='GPT_SoVITS/pretrained_models/chinese-roberta-wwm-ext-large', help='Path to the BERT model')
parser.add_argument('--infer_ttswebui', type=int, default=9872, help='Port number for inference server')
parser.add_argument('--is_half', action='store_true', help='Enable half precision')
if os.path.exists("./gweight.txt"):
    with open("./gweight.txt", 'r', encoding="utf-8") as file:
        gweight_data = file.read()
        gpt_path = os.environ.get(
            "gpt_path", gweight_data)
else:
    gpt_path = os.environ.get(
        "gpt_path",
"GPT_SoVITS/pretrained_models/s1bert25hz-2kh-longer-epoch=68e-step=50232.ckpt")
```

```
if os.path.exists("./sweight.txt"):
    with open("./sweight.txt", 'r', encoding="utf-8") as file:
        sweight_data = file.read()
        sovits_path = os.environ.get("sovits_path", sweight_data)
else:
    sovits_path = os.environ.get("sovits_path", "GPT_SoVITS/pretrained_models/s2G488k.pth")

cnhubert_base_path = os.environ.get(
    "cnhubert_base_path", "GPT_SoVITS/pretrained_models/chinese-hubert-base"
)
bert_path = os.environ.get(
    "bert_path", "GPT_SoVITS/pretrained_models/chinese-roberta-wwm-ext-large"
)
infer_ttswebui = os.environ.get("infer_ttswebui", 9872)
infer_ttswebui = int(infer_ttswebui)
is_share = os.environ.get("is_share", "False")
is_share = eval(is_share)
if "_CUDA_VISIBLE_DEVICES" in os.environ:
    os.environ["CUDA_VISIBLE_DEVICES"] = os.environ["_CUDA_VISIBLE_DEVICES"]
is_half = eval(os.environ.get("is_half", "True")) and torch.cuda.is_available()
punctuation = set(['!', '?', '…', ',', '.', '-'," "])
import gradio as gr
from transformers import AutoModelForMaskedLM, AutoTokenizer
import numpy as np
import librosa
from feature_extractor import cnhubert

cnhubert.cnhubert_base_path = cnhubert_base_path

from module.models import SynthesizerTrn
from AR.models.t2s_lightning_module import Text2SemanticLightningModule
from text import cleaned_text_to_sequence
from text.cleaner import clean_text
from time import time as ttime
from module.mel_processing import spectrogram_torch
from tools.my_utils import load_audio
from tools.i18n.i18n import I18nAuto

i18n = I18nAuto()

# os.environ['PYTORCH_ENABLE_MPS_FALLBACK'] = '1'  # 确保直接启动推理 UI 时也能够设置。

if torch.cuda.is_available():
    device = "cuda"
```

```
else:
    device = "cpu"

tokenizer = AutoTokenizer.from_pretrained(bert_path)
bert_model = AutoModelForMaskedLM.from_pretrained(bert_path)
if is_half == True:
    bert_model = bert_model.half().to(device)
else:
    bert_model = bert_model.to(device)


def get_bert_feature(text, word2ph):
    with torch.no_grad():
        inputs = tokenizer(text, return_tensors="pt")
        for i in inputs:
            inputs[i] = inputs[i].to(device)
        res = bert_model(**inputs, output_hidden_states=True)
        res = torch.cat(res["hidden_states"][-3:-2], -1)[0].cpu()[1:-1]
    assert len(word2ph) == len(text)
    phone_level_feature = []
    for i in range(len(word2ph)):
        repeat_feature = res[i].repeat(word2ph[i], 1)
        phone_level_feature.append(repeat_feature)
    phone_level_feature = torch.cat(phone_level_feature, dim=0)
    return phone_level_feature.T


class DictToAttrRecursive(dict):
    def __init__(self, input_dict):
        super().__init__(input_dict)
        for key, value in input_dict.items():
            if isinstance(value, dict):
                value = DictToAttrRecursive(value)
            self[key] = value
            setattr(self, key, value)

    def __getattr__(self, item):
        try:
            return self[item]
        except KeyError:
            raise AttributeError(f"Attribute {item} not found")

    def __setattr__(self, key, value):
        if isinstance(value, dict):
            value = DictToAttrRecursive(value)
```

```
        super(DictToAttrRecursive, self).__setitem__(key, value)
        super().__setattr__(key, value)

    def __delattr__(self, item):
        try:
            del self[item]
        except KeyError:
            raise AttributeError(f"Attribute {item} not found")


ssl_model = cnhubert.get_model()
if is_half == True:
    ssl_model = ssl_model.half().to(device)
else:
    ssl_model = ssl_model.to(device)


def change_sovits_weights(sovits_path):
    global vq_model, hps
    dict_s2 = torch.load(sovits_path, map_location="cpu")
    hps = dict_s2["config"]
    hps = DictToAttrRecursive(hps)
    hps.model.semantic_frame_rate = "25hz"
    vq_model = SynthesizerTrn(
        hps.data.filter_length // 2 + 1,
        hps.train.segment_size // hps.data.hop_length,
        n_speakers=hps.data.n_speakers,
        **hps.model
    )
    if ("pretrained" not in sovits_path):
        del vq_model.enc_q
    if is_half == True:
        vq_model = vq_model.half().to(device)
    else:
        vq_model = vq_model.to(device)
    vq_model.eval()
    print(vq_model.load_state_dict(dict_s2["weight"], strict=False))
    with open("./sweight.txt", "w", encoding="utf-8") as f:
        f.write(sovits_path)


change_sovits_weights(sovits_path)


def change_gpt_weights(gpt_path):
```

```
    global hz, max_sec, t2s_model, config
    hz = 50
    dict_s1 = torch.load(gpt_path, map_location="cpu")
    config = dict_s1["config"]
    max_sec = config["data"]["max_sec"]
    t2s_model = Text2SemanticLightningModule(config, "****", is_train=False)
    t2s_model.load_state_dict(dict_s1["weight"])
    if is_half == True:
        t2s_model = t2s_model.half()
    t2s_model = t2s_model.to(device)
    t2s_model.eval()
    total = sum([param.nelement() for param in t2s_model.parameters()])
    print("Number of parameter: %.2fM" % (total / 1e6))
    with open("./gweight.txt", "w", encoding="utf-8") as f: f.write(gpt_path)


change_gpt_weights(gpt_path)


def get_spepc(hps, filename):
    audio = load_audio(filename, int(hps.data.sampling_rate))
    audio = torch.FloatTensor(audio)
    audio_norm = audio
    audio_norm = audio_norm.unsqueeze(0)
    spec = spectrogram_torch(
        audio_norm,
        hps.data.filter_length,
        hps.data.sampling_rate,
        hps.data.hop_length,
        hps.data.win_length,
        center=False,
    )
    return spec


def clean_text_inf(text, language):
    phones, word2ph, norm_text = clean_text(text, language)
    phones = cleaned_text_to_sequence(phones)
    return phones, word2ph, norm_text

dtype=torch.float16 if is_half == True else torch.float32
def get_bert_inf(phones, word2ph, norm_text, language):
    language=language.replace("all_","")
    if language == "zh":
        bert = get_bert_feature(norm_text, word2ph).to(device)#.to(dtype)
```

```
    else:
        bert = torch.zeros(
            (1024, len(phones)),
            dtype=torch.float16 if is_half == True else torch.float32,
        ).to(device)

    return bert


splits = {", ", "。", "？", "！", ",", ".", "?", "!", "~", ":", "：", "—", "…", }


def get_first(text):
    pattern = "[" + "".join(re.escape(sep) for sep in splits) + "]"
    text = re.split(pattern, text)[0].strip()
    return text


def get_phones_and_bert(text,language):
    if language in {"en","all_zh","all_ja"}:
        language = language.replace("all_","")
        if language == "en":
            LangSegment.setfilters(["en"])
            formattext = " ".join(tmp["text"] for tmp in LangSegment.getTexts(text))
        else:
            # 因无法区别中日文汉字,以用户输入为准
            formattext = text
        while "  " in formattext:
            formattext = formattext.replace("  ", " ")
        phones, word2ph, norm_text = clean_text_inf(formattext, language)
        if language == "zh":
            bert = get_bert_feature(norm_text, word2ph).to(device)
        else:
            bert = torch.zeros(
                (1024, len(phones)),
                dtype=torch.float16 if is_half == True else torch.float32,
            ).to(device)
    elif language in {"zh", "ja","auto"}:
        textlist=[]
        langlist=[]
        LangSegment.setfilters(["zh","ja","en","ko"])
        if language == "auto":
            for tmp in LangSegment.getTexts(text):
                if tmp["lang"] == "ko":
                    langlist.append("zh")
```

```
                    textlist.append(tmp["text"])
                else:
                    langlist.append(tmp["lang"])
                    textlist.append(tmp["text"])
        else:
            for tmp in LangSegment.getTexts(text):
                if tmp["lang"] == "en":
                    langlist.append(tmp["lang"])
                else:
                    # 因无法区别中日文汉字,以用户输入为准
                    langlist.append(language)
                textlist.append(tmp["text"])
        print(textlist)
        print(langlist)
        phones_list = []
        bert_list = []
        norm_text_list = []
        for i in range(len(textlist)):
            lang = langlist[i]
            phones, word2ph, norm_text = clean_text_inf(textlist[i], lang)
            bert = get_bert_inf(phones, word2ph, norm_text, lang)
            phones_list.append(phones)
            norm_text_list.append(norm_text)
            bert_list.append(bert)
        bert = torch.cat(bert_list, dim=1)
        phones = sum(phones_list, [])
        norm_text = ''.join(norm_text_list)

    return phones,bert.to(dtype),norm_text


def merge_short_text_in_array(texts, threshold):
    if (len(texts)) < 2:
        return texts
    result = []
    text = ""
    for ele in texts:
        text += ele
        if len(text) >= threshold:
            result.append(text)
            text = ""
    if (len(text) > 0):
        if len(result) == 0:
            result.append(text)
```

```
        else:
            result[len(result) - 1] += text
    return result

def get_tts_wav(ref_wav_path, text, prompt_text = None, prompt_language = 'all_zh',  text_language = 'all_zh', how_to_cut=i18n("不切"), top_k=5, top_p=1, temperature=1, ref_free = True):
    #PLUS
    ref_free = True
    prompt_text = None
    how_to_cut=i18n("不切")
    #PLUS
    if prompt_text is None or len(prompt_text) == 0:
        ref_free = True
    t0 = ttime()

    prompt_language = 'all_zh'
    text_language = 'all_zh'
    text = text.strip("\n")
    text = replace_consecutive_punctuation(text)
    if (text[0] not in splits and len(get_first(text)) < 4): text = "。" + text if text_language != "en" else "." + text

    print(i18n("实际输入的目标文本:"), text)
    zero_wav = np.zeros(
        int(hps.data.sampling_rate * 0.3),
        dtype=np.float16 if is_half == True else np.float32,
    )
    t1 = ttime()

    while "\n\n" in text:
        text = text.replace("\n\n", "\n")
    print(i18n("实际输入的目标文本(切句后):"), text)
    texts = text.split("\n")
    texts = process_text(texts)
    texts = merge_short_text_in_array(texts, 5)
    audio_opt = []
    if not ref_free:
        phones1,bert1,norm_text1=get_phones_and_bert(prompt_text, prompt_language)
    for text in texts:
        # 解决输入目标文本的空行导致报错的问题
        if (len(text.strip()) == 0):
            continue
        if (text[-1] not in splits): text += "。" if text_language != "en" else "."
        print(i18n("实际输入的目标文本(每句):"), text)
```

```
        phones2,bert2,norm_text2=get_phones_and_bert(text, text_language)
        print(i18n("前端处理后的文本(每句):"), norm_text2)
        if not ref_free:
            bert = torch.cat([bert1, bert2], 1)
            all_phoneme_ids = torch.LongTensor(phones1+phones2).to(device).unsqueeze(0)
        else:
            bert = bert2
            all_phoneme_ids = torch.LongTensor(phones2).to(device).unsqueeze(0)

        bert = bert.to(device).unsqueeze(0)
        all_phoneme_len = torch.tensor([all_phoneme_ids.shape[-1]]).to(device)

        t2 = ttime()
        with torch.no_grad():
            # pred_semantic = t2s_model.model.infer(
            pred_semantic, idx = t2s_model.model.infer_panel(
                all_phoneme_ids,
                all_phoneme_len,
                None if ref_free else prompt,
                bert,
                # prompt_phone_len=ph_offset,
                top_k=top_k,
                top_p=top_p,
                temperature=temperature,
                early_stop_num=hz * max_sec,
            )
        t3 = ttime()
        # print(pred_semantic.shape,idx)
        pred_semantic = pred_semantic[:, -idx:].unsqueeze(
            0
        )  # .unsqueeze(0)#mq 要多 unsqueeze 一次
        refer = get_spepc(hps, ref_wav_path)  # .to(device)
        if is_half == True:
            refer = refer.half().to(device)
        else:
            refer = refer.to(device)
        # audio = vq_model.decode(pred_semantic, all_phoneme_ids, refer).detach().cpu().numpy()[0,
0]
        audio = (
            vq_model.decode(
                pred_semantic, torch.LongTensor(phones2).to(device).unsqueeze(0), refer
            )
                .detach()
                .cpu()
```

```
                .numpy()[0, 0]
            )  ###试试重建不带上 prompt 部分
            max_audio=np.abs(audio).max()#简单防止 16bit 爆音
            if max_audio>1:audio/=max_audio
            audio_opt.append(audio)
            audio_opt.append(zero_wav)
            t4 = ttime()
        print("%.3f\t%.3f\t%.3f\t%.3f" % (t1 - t0, t2 - t1, t3 - t2, t4 - t3))
        yield hps.data.sampling_rate, (np.concatenate(audio_opt, 0) * 32768).astype(
            np.int16
        )
        audio_data2 = (np.concatenate(audio_opt, 0) * 32768).astype(np.int16)
        # 将音频数据写入本地文件
        output_file = '../Result/Generation_Audio.wav'  # 指定输出文件名
        sampling_rate = hps.data.sampling_rate  # 获取采样率
        write_wav_file(output_file, sampling_rate, audio_data2)


def split(todo_text):
    todo_text = todo_text.replace("……", "。").replace("——", ", ")
    if todo_text[-1] not in splits:
        todo_text += "。"
    i_split_head = i_split_tail = 0
    len_text = len(todo_text)
    todo_texts = []
    while 1:
        if i_split_head >= len_text:
            break  # 结尾一定有标点，所以直接跳出即可，最后一段在上次已加入
        if todo_text[i_split_head] in splits:
            i_split_head += 1
            todo_texts.append(todo_text[i_split_tail:i_split_head])
            i_split_tail = i_split_head
        else:
            i_split_head += 1
    return todo_texts


def custom_sort_key(s):
    # 使用正则表达式提取字符串中的数字部分和非数字部分
    parts = re.split('(\d+)', s)
    # 将数字部分转换为整数，非数字部分保持不变
    parts = [int(part) if part.isdigit() else part for part in parts]
    return parts

def process_text(texts):
```

```
    _text=[]
    if all(text in [None, " ", "\n",""] for text in texts):
        raise ValueError(i18n("请输入有效文本"))
    for text in texts:
        if text in  [None, " ", ""]:
            pass
        else:
            _text.append(text)
    return _text


def replace_consecutive_punctuation(text):
    punctuations = ''.join(re.escape(p) for p in punctuation)
    pattern = f'([{punctuations}])([{punctuations}])+'
    result = re.sub(pattern, r'\1', text)
    return result


def change_choices():
    SoVITS_names, GPT_names = get_weights_names()
    return {"choices": sorted(SoVITS_names, key=custom_sort_key), "__type__": "update"},
{"choices": sorted(GPT_names, key=custom_sort_key), "__type__": "update"}


pretrained_sovits_name = "GPT_SoVITS/pretrained_models/s2G488k.pth"
pretrained_gpt_name =
"GPT_SoVITS/pretrained_models/s1bert25hz-2kh-longer-epoch=68e-step=50232.ckpt"
SoVITS_weight_root = "SoVITS_weights"
GPT_weight_root = "GPT_weights"
os.makedirs(SoVITS_weight_root, exist_ok=True)
os.makedirs(GPT_weight_root, exist_ok=True)


def get_weights_names():
    SoVITS_names = [pretrained_sovits_name]
    for name in os.listdir(SoVITS_weight_root):
        if name.endswith(".pth"): SoVITS_names.append("%s/%s" % (SoVITS_weight_root, name))
    GPT_names = [pretrained_gpt_name]
    for name in os.listdir(GPT_weight_root):
        if name.endswith(".ckpt"): GPT_names.append("%s/%s" % (GPT_weight_root, name))
    return SoVITS_names, GPT_names


SoVITS_names, GPT_names = get_weights_names()
def main(args):
```

```
    # 加载 SoVITS 和 GPT 模型权重
    sovits_path = r"D:\大学\大三上\人工智能应用实践
\Project\Audio_trans\GPT-SoVITS-beta\GPT-SoVITS-beta0706\SoVITS_weights\ShenLiModel_e12_s144.p
th"
    gpt_path = r"D:\大学\大三上\人工智能应用实践
\Project\Audio_trans\GPT-SoVITS-beta\GPT-SoVITS-beta0706\GPT_weights\ShenLiModel-e15.ckpt"
    change_sovits_weights(sovits_path)
    change_gpt_weights(gpt_path)
    # 处理输入文本
    args.text = args.text.strip()

    # 执行文本到语音的推理任务
    try:
        gen = get_tts_wav(
            args.ref_wav_path,
            args.text,
        )

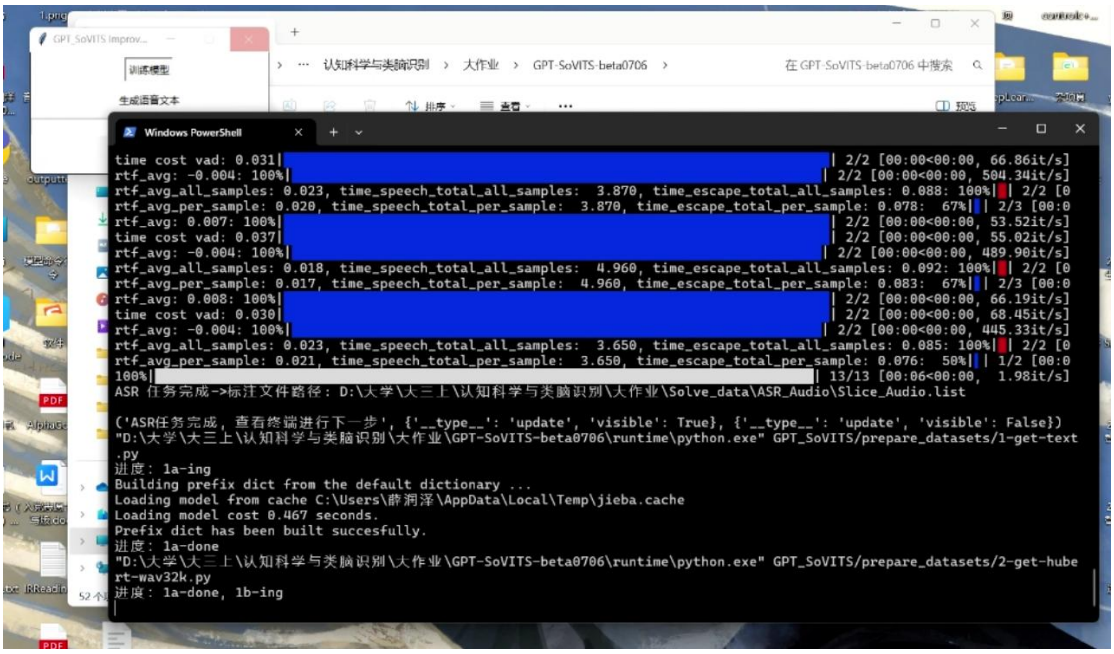
        # 遍历生成器的所有值
        while True:
            # print("YES")
            sample_rate, audio_data = next(gen)
            # 这里你可以处理 sample_rate 和 audio_data，比如保存到文件
            # output_file = 'output.wav'
            # with open(output_file, 'ab') as f:  # 使用'ab'模式追加写入
            #     f.write(audio_data.tobytes())
    except StopIteration:
        # 生成器没有更多的值了
        print(f"Audio generation completed.")
if __name__ == '__main__':
    # 解析命令行参数
    # 执行主函数
    args = parser.parse_args()
    main(args)
```

六、实验结果

成功完成了 GPT-SoVITS 项目的接口适配工作，分割后的各模块

功能重新组合运行正常。实验过程中，解决了 Pytorch 架构下更换接口时遇到的本机环境问题，验证了 GPT-SoVITS 的各项功能，包括声音伴奏分离、自动训练集分割、中文自动语音识别(ASR)和文本标注等。实验结果表明，GPT-SoVITS 系统具有较高的声音相似度和真实感。且接口适配成功，各模块功能正常。

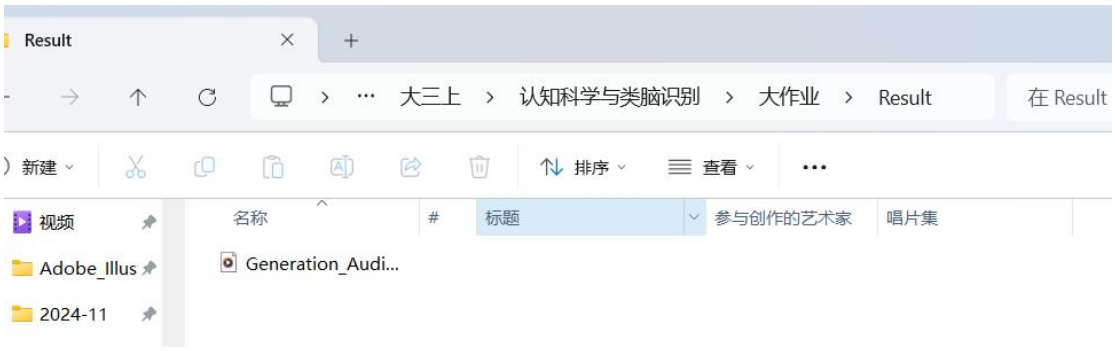
下图为模型训练的一部分：



下述为我写的一个交互页面，此时完成了模型训练，进行 tts 推理：



生成的结果存入到 Result 文件夹中



七、实验心得

通过本次实验，我实际参与到大型的神经网络训练流程中，手动去操作体会类脑知识。同时接口开发工作让我能实际研究各个模块的代码，改写各部分代码。在本次实验中，我掌握了 GPT-SoVITS 的基本原理和工作流程，了解到语音合成的一种模型实现形式。